

# DSA Patterns & Algorithms

The patterns behind most online-test and interview problems. Each section tells you how to recognise the pattern, explains the idea in plain words, and then walks through worked examples: a full C++ program, a real input, and the **real output** from running it.

---

- |           |                               |           |                                 |
|-----------|-------------------------------|-----------|---------------------------------|
| <b>1</b>  | How to spot the pattern       | <b>2</b>  | Two pointers                    |
| <b>3</b>  | Sliding window                | <b>4</b>  | Prefix sums & difference arrays |
| <b>5</b>  | Binary search & on the answer | <b>6</b>  | Monotonic stack                 |
| <b>7</b>  | Greedy                        | <b>8</b>  | Recursion & backtracking        |
| <b>9</b>  | Linked lists                  | <b>10</b> | Trees                           |
| <b>11</b> | Graphs                        | <b>12</b> | Union-find (DSU)                |
| <b>13</b> | Dynamic programming           | <b>14</b> | Heaps                           |
| <b>15</b> | String algorithms             | <b>16</b> | Trie                            |
| <b>17</b> | Fenwick & segment tree        | <b>18</b> | Maths for coding tests          |
| <b>19</b> | Quick revision tables         |           |                                 |

**How to use these notes:** for each example, cover the code, read only the problem, and try to solve it in 15 minutes. Then compare with the code. Every program here compiles with g++ (C++17); the first two lines (`#include <bits/stdc++.h>` and `using namespace std;`) are left out to save paper. This pairs with the C++ *STL Notes*, which cover the containers used here.

## 1 How to spot the pattern

Most OA problems are one of about 20 patterns in disguise. Before coding, read the problem and look for these clues. If a clue matches, you already know the shape of the solution.

| If the problem says or has...                                       | Think of                                           |
|---------------------------------------------------------------------|----------------------------------------------------|
| sorted array, find a pair / triplet                                 | Two pointers (§2)                                  |
| contiguous subarray or substring, longest / shortest / count        | Sliding window (§3) or prefix sums (§4)            |
| many range-sum queries, or "sum equals k" with negatives            | Prefix sums + hash map (§4)                        |
| many range <i>updates</i> , then read the array once                | Difference array (§4)                              |
| sorted data, or "minimum X such that..." / "maximum X such that..." | Binary search, or binary search on the answer (§5) |
| next greater / smaller, span, histogram                             | Monotonic stack (§6)                               |
| choose the best option each step, intervals, scheduling             | Greedy, usually after sorting (§7)                 |
| "all" subsets / permutations / combinations, $n \leq 20$            | Backtracking (§8) or bitmasks                      |
| linked list, cycle, middle                                          | Fast and slow pointers (§9)                        |
| tree: depth, path, ancestor                                         | DFS recursion on trees (§10)                       |
| grid, network, dependencies, shortest steps                         | BFS / DFS / topological sort (§11)                 |
| weighted shortest path                                              | Dijkstra (§11)                                     |
| groups that merge, "are these connected?"                           | Union-find (§12)                                   |
| count ways, min/max cost, choices that affect later choices         | Dynamic programming (§13)                          |
| running median, top K, always take the smallest                     | Heap (§14)                                         |
| find a pattern in text, palindromes                                 | String algorithms (§15)                            |
| prefix searches on many words                                       | Trie (§16)                                         |
| range queries <i>and</i> updates mixed together                     | Fenwick tree / segment tree (§17)                  |
| answer modulo $10^9+7$ , primes, $nCr$                              | Maths (§18)                                        |

### A 5-step routine for any new problem

1. **Read the constraints first.**  $n \leq 10^5$  rules out  $O(n^2)$ . This alone often tells you the pattern (see the STL notes, section 3).
2. **Solve the sample by hand.** Write down what you did. Your hand method is usually the algorithm.
3. **Think of the brute force**, then ask: what work am I repeating? Removing that repeated work is where patterns come in.
4. **Match the clues** against the table above.
5. **Before coding, test your idea** on an edge case:  $n = 1$ , all equal, all negative, already sorted.

## 2 Two pointers

### Spot it when...

- the array is sorted (or you can sort it)
- find a pair or triplet with a target sum
- compare from both ends (palindromes, containers)
- remove duplicates or partition in place

### The idea

Put one pointer at each end (or both at the start). Move the pointer that can improve the answer. Each pointer only moves forward, so the whole scan is  $O(n)$  instead of  $O(n^2)$ .

### EXAMPLE All pairs with a given sum (sorted array)

Print every pair of values in the sorted array that adds up to the target.

```
int main() {
    int n, target; cin >> n >> target;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    int i = 0, j = n - 1;
    while (i < j) {
        int s = a[i] + a[j];
        if (s == target) { cout << a[i] << " " << a[j] << "\n"; i++; j--; }
        else if (s < target) i++;           // need a bigger sum
        else j--;                           // need a smaller sum
    }
}
```

#### INPUT

```
6 10
1 2 3 4 6 8
```

#### OUTPUT

```
2 8
4 6
```

**How it works:** if the sum is too small, only moving `i` right can make it bigger. If it's too big, only moving `j` left can make it smaller.

**Time:**  $O(n)$

**EXAMPLE 3Sum: triplets that add to zero**

Print every unique triplet (no repeats) whose sum is 0.

```
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;    // skip duplicate first numbers
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                cout << a[i] << " " << a[l] << " " << a[r] << "\n";
                while (l < r && a[l] == a[l + 1]) l++;    // skip duplicates
                while (l < r && a[r] == a[r - 1]) r--;
                l++; r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
}
```

**INPUT**

```
6
-1 0 1 2 -1 -4
```

**OUTPUT**

```
-1 -1 2
-1 0 1
```

**How it works:** fix the first number, then run the pair-sum two-pointer scan on the rest. Skipping equal neighbours prevents repeated triplets.

**Time:**  $O(n^2)$

**EXAMPLE Container with most water**

Lines of height  $h[i]$  stand at positions  $i$ . Pick two lines that hold the most water (width  $\times$  shorter height).

```
int main() {
    int n; cin >> n;
    vector<int> h(n);
    for (auto &x : h) cin >> x;
    int i = 0, j = n - 1;
    long long best = 0;
    while (i < j) {
        best = max(best, 1LL * (j - i) * min(h[i], h[j]));
        if (h[i] < h[j]) i++;           // the shorter line limits us: move it
        else j--;
    }
    cout << best << "\n";
}
```

**INPUT**

```
9
1 8 6 2 5 4 8 3 7
```

**OUTPUT**

```
49
```

**How it works:** the width only shrinks as pointers move, so the only hope of a bigger area is a taller short side. Moving the taller side can never help.

**EXAMPLE Remove duplicates from a sorted array, in place**

Keep one copy of each value at the front. Print how many are left and the values.

```
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    int k = 0;                                     // a[0..k-1] holds the unique values
    for (int i = 0; i < n; i++)
        if (k == 0 || a[i] != a[k - 1]) a[k++] = a[i];
    cout << k << "\n";
    for (int i = 0; i < k; i++) cout << a[i] << " ";
    cout << "\n";
}
```

**INPUT**

```
10
0 0 1 1 1 2 2 3 3 4
```

**OUTPUT**

```
5
0 1 2 3 4
```

**How it works:** a slow pointer  $k$  marks where the next new value goes; a fast pointer  $i$  reads ahead.

**Time:**  $O(n)$ , no extra memory

### 3 Sliding window

#### Spot it when...

- contiguous subarray or substring
- longest / shortest / count of windows with some property
- the property only gets "worse" as the window grows (e.g. sum of positives, number of distinct characters)

#### The idea

Keep a window `[left, right]`. Grow it by moving `right`. When the window breaks the rule, shrink it from the `left` until it's valid again. Each index enters and leaves once, so it's  $O(n)$ .

```
// Variable-size window template
int left = 0;
for (int right = 0; right < n; right++) {
    add a[right] to the window;
    while (window is not valid) { remove a[left] from the window; left++; }
    update the answer using the window [left, right];
}
```

#### EXAMPLE Fixed window: maximum sum of k consecutive numbers

Find the largest sum of any k numbers in a row.

```
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    long long sum = 0, best = LLONG_MIN;
    for (int i = 0; i < n; i++) {
        sum += a[i]; // new element enters
        if (i >= k) sum -= a[i - k]; // old element leaves
        if (i >= k - 1) best = max(best, sum);
    }
    cout << best << "\n";
}
```

#### INPUT

```
6 3
2 1 5 1 3 2
```

#### OUTPUT

```
9
```

**How it works:** instead of re-adding k numbers for each window, add the one that enters and subtract the one that leaves.

**Time:**  $O(n)$

**EXAMPLE Longest substring without repeating characters**

For each word, print the length of the longest substring with all different characters.

```
int main() {
    string s;
    while (cin >> s) {
        vector<int> last(256, -1); // last index where each character was seen
        int left = 0, best = 0;
        for (int right = 0; right < (int)s.size(); right++) {
            unsigned char ch = s[right];
            if (last[ch] >= left) left = last[ch] + 1; // jump past the repeat
            last[ch] = right;
            best = max(best, right - left + 1);
        }
        cout << s << " -> " << best << "\n";
    }
}
```

**INPUT**

```
abcabcbb
bbbbbb
pwwkew
```

**OUTPUT**

```
abcabcbb -> 3
bbbbbb -> 1
pwwkew -> 3
```

**How it works:** when a character repeats inside the window, move `left` just past its previous position.

**EXAMPLE Shortest subarray with sum  $\geq$  target (positive numbers)**

Find the length of the shortest contiguous subarray whose sum is at least the target, or 0.

```
int main() {
    int n, target; cin >> n >> target;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    int left = 0, best = INT_MAX;
    long long sum = 0;
    for (int right = 0; right < n; right++) {
        sum += a[right];
        while (sum >= target) { // valid: try to make it shorter
            best = min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    cout << (best == INT_MAX ? 0 : best) << "\n";
}
```

**INPUT**

```
6 7
2 3 1 2 4 3
```

**OUTPUT**

```
2
```

**How it works:** this only works because all numbers are positive, so shrinking always lowers the sum. With negatives, use prefix sums instead.

**Time:**  $O(n)$

**EXAMPLE Longest run of 1s if you may flip k zeros**

You may change at most k zeros into ones. What's the longest block of ones you can get?

```
int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    int left = 0, zeros = 0, best = 0;
    for (int right = 0; right < n; right++) {
        if (a[right] == 0) zeros++;
        while (zeros > k) {           // too many zeros: shrink
            if (a[left] == 0) zeros--;
            left++;
        }
        best = max(best, right - left + 1);
    }
    cout << best << "\n";
}
```

**INPUT**

```
11 2
1 1 1 0 0 0 1 1 1 1 0
```

**OUTPUT**

```
6
```

**How it works:** rephrase it as "longest window with at most k zeros", which is a standard sliding window.



### EXAMPLE Count subarrays with exactly K distinct numbers

Count the contiguous subarrays that contain exactly K different values.

```
long long atMost(vector<int>& a, int k) {    // subarrays with <= k distinct values
    unordered_map<int,int> cnt;
    long long res = 0;
    int left = 0;
    for (int right = 0; right < (int)a.size(); right++) {
        if (cnt[a[right]]++ == 0) k--;
        while (k < 0) { if (--cnt[a[left]] == 0) k++; left++; }
        res += right - left + 1;           // all windows ending at right
    }
    return res;
}

int main() {
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    cout << atMost(a, k) - atMost(a, k - 1) << "\n";
}
```

INPUT

```
5 2
1 2 1 2 3
```

OUTPUT

```
7
```

**How it works:** "exactly K" is hard to slide directly, but "at most K" is easy. Exactly  $K = \text{atMost}(K) - \text{atMost}(K-1)$ . This trick shows up a lot.

## 4 Prefix sums & difference arrays

### Spot it when...

- many "sum from l to r" queries
- count subarrays with sum = k (negatives allowed)
- "divisible by k" subarrays
- many range updates (add v to l..r), then print the array

### The idea

**Prefix sum:**  $\text{pre}[i] = \text{sum of the first } i \text{ elements}$ , so  $\text{sum}(l..r) = \text{pre}[r+1] - \text{pre}[l]$ . **Difference array:** to add v to l..r, do  $d[l] += v$  and  $d[r+1] -= v$ ; one prefix sum at the end gives the final array.

**EXAMPLE 2D prefix sums: rectangle sum queries**

Answer queries asking for the sum of a rectangle (r1, c1) to (r2, c2), 0-indexed.

```
int main() {
    int n, m, q; cin >> n >> m >> q;
    vector<vector<long long>> P(n + 1, vector<long long>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            long long x; cin >> x;
            P[i][j] = x + P[i-1][j] + P[i][j-1] - P[i-1][j-1];
        }
    while (q--) {
        int r1, c1, r2, c2; cin >> r1 >> c1 >> r2 >> c2;
        r1++; c1++; r2++; c2++; // shift to 1-indexed
        cout << P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1] << "\n";
    }
}
```

**INPUT**

```
3 3 2
1 2 3
4 5 6
7 8 9
1 1 2 2
0 0 2 2
```

**OUTPUT**

```
28
45
```

**How it works:**  $P[i][j]$  holds the sum of the top-left rectangle. Any rectangle = big – top strip – left strip + the corner that was subtracted twice.

**Time:**  $O(nm)$  setup,  $O(1)$  per query

**EXAMPLE Difference array: many range additions**

Start with  $n$  zeros. Each update adds  $v$  to every index from  $l$  to  $r$ . Print the final array.

```
int main() {
    int n, q; cin >> n >> q;
    vector<long long> d(n + 1, 0);
    while (q--) {
        int l, r; long long v; cin >> l >> r >> v;
        d[l] += v;           // start adding at l
        d[r + 1] -= v;       // stop after r
    }
    long long run = 0;
    for (int i = 0; i < n; i++) { run += d[i]; cout << run << " "; }
    cout << "\n";
}
```

**INPUT**

```
5 3
0 2 3
1 4 2
2 2 -1
```

**OUTPUT**

```
3 5 4 2 2
```

**How it works:** each update is  $O(1)$ . The running sum at the end "spreads" every update over its range.

**Time:**  $O(n + q)$

**EXAMPLE Longest subarray with sum k (negatives allowed)**

Find the length of the longest contiguous subarray whose sum is exactly  $k$ .

```
int main() {
    int n; long long k; cin >> n >> k;
    unordered_map<long long, int> first; // prefix sum -> first index where it appeared
    first[0] = -1;
    long long sum = 0; int best = 0;
    for (int i = 0; i < n; i++) {
        long long x; cin >> x;
        sum += x;
        if (first.count(sum - k)) best = max(best, i - first[sum - k]);
        if (!first.count(sum)) first[sum] = i; // keep the EARLIEST index only
    }
    cout << best << "\n";
}
```

**INPUT**

```
5 3
1 -1 5 -2 3
```

**OUTPUT**

```
4
```

**How it works:** if  $\text{prefix}[i] - \text{prefix}[j] = k$ , the subarray  $j+1..i$  sums to  $k$ . To make it long, remember the earliest  $j$  for each prefix value.

### EXAMPLE Count subarrays whose sum is divisible by k

Count the contiguous subarrays with a sum divisible by k.

```
int main() {
    int n, k; cin >> n >> k;
    vector<long long> cnt(k, 0);           // how many prefixes have each remainder
    cnt[0] = 1;
    long long sum = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        long long x; cin >> x;
        sum += x;
        int r = ((sum % k) + k) % k;       // fix negative remainders
        ans += cnt[r];
        cnt[r]++;
    }
    cout << ans << "\n";
}
```

INPUT

```
6 5
4 5 0 -2 -3 1
```

OUTPUT

```
7
```

**How it works:** two prefixes with the same remainder mod k differ by a multiple of k. So count pairs of equal remainders.

## 5 Binary search & binary search on the answer

### Spot it when...

- the array is sorted, or rotated sorted
- "find the minimum X such that..." or "maximum X such that..."
- checking one candidate answer is easy, but finding the best one directly is hard
- the answer range is huge (up to  $10^9$  or  $10^{18}$ )

### The idea

If a yes/no check is *monotonic* (false, false, ..., true, true), you can binary search for the first true. Each step halves the range, so even  $10^{18}$  takes only about 60 steps.

```
// First value in [lo, hi] where ok(x) is true. Requires: false...false true...true
long long lo = LOW, hi = HIGH;
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid; else lo = mid + 1;
}
// lo is the answer
```

**EXAMPLE Classic binary search**

For each query, print the index of x in the sorted array, or -1.

```
int search(vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // avoids overflow of lo + hi
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

int main() {
    int n, q; cin >> n >> q;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    while (q--) { int x; cin >> x; cout << search(a, x) << "\n"; }
}
```

**INPUT**

```
6 2
1 3 5 7 9 11
7 4
```

**OUTPUT**

```
3
-1
```

**Time:**  $O(\log n)$  per query

**EXAMPLE Search in a rotated sorted array**

A sorted array was rotated (e.g. 0 1 2 4 5 6 7 became 4 5 6 7 0 1 2). Find the index of the target.

```
int main() {
    int n, q; cin >> n >> q;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    while (q--) {
        int t; cin >> t;
        int lo = 0, hi = n - 1, ans = -1;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            if (a[mid] == t) { ans = mid; break; }
            if (a[lo] <= a[mid]) {                // left half is sorted
                if (a[lo] <= t && t < a[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {                             // right half is sorted
                if (a[mid] < t && t <= a[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        cout << ans << "\n";
    }
}
```

**INPUT**

```
7 2
4 5 6 7 0 1 2
0 3
```

**OUTPUT**

```
4
-1
```

**How it works:** at any mid, at least one half is properly sorted. Check if the target lies in that sorted half; if not, it must be in the other half.

**EXAMPLE Koko eating bananas (binary search on the answer)**

Piles of bananas; each hour Koko eats up to  $s$  bananas from one pile. Find the smallest speed  $s$  to finish within  $h$  hours.

```
int main() {
    int n; long long h; cin >> n >> h;
    vector<long long> p(n);
    for (auto &x : p) cin >> x;
    auto ok = [&](long long s) {          // can she finish at speed s?
        long long hours = 0;
        for (long long x : p) hours += (x + s - 1) / s;    // ceil(x / s)
        return hours <= h;
    };
    long long lo = 1, hi = *max_element(p.begin(), p.end());
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid; else lo = mid + 1;
    }
    cout << lo << "\n";
}
```

**INPUT**

```
4 8
3 6 7 11
```

**OUTPUT**

```
4
```

**How it works:** a faster speed never makes things worse, so  $ok(s)$  is false...false, true...true. Binary search finds the first true.

**Time:**  $O(n \log(\max \text{ pile}))$

### EXAMPLE Aggressive cows (maximise the minimum distance)

Place  $c$  cows in stalls so that the smallest distance between any two cows is as large as possible.

```
int main() {
    int n, cows; cin >> n >> cows;
    vector<long long> s(n);
    for (auto &x : s) cin >> x;
    sort(s.begin(), s.end());
    auto ok = [&](long long d) { // can we keep every gap >= d?
        int placed = 1; long long last = s[0];
        for (int i = 1; i < n; i++)
            if (s[i] - last >= d) { placed++; last = s[i]; }
        return placed >= cows;
    };
    long long lo = 0, hi = s[n - 1] - s[0];
    while (lo < hi) { // LAST d where ok(d) is true
        long long mid = lo + (hi - lo + 1) / 2; // round UP to avoid an endless loop
        if (ok(mid)) lo = mid; else hi = mid - 1;
    }
    cout << lo << "\n";
}
```

INPUT

```
5 3
1 2 4 8 9
```

OUTPUT

```
3
```

**How it works:** here the check is true...true, false...false, so we search for the *last* true. Note the `+ 1` in mid, otherwise `lo = mid` could loop forever.

### EXAMPLE Integer square root without sqrt()

Print the largest integer  $r$  with  $r \times r \leq x$ .

```
int main() {
    long long x;
    while (cin >> x) {
        long long lo = 0, hi = 2000000000LL;
        while (lo < hi) {
            long long mid = lo + (hi - lo + 1) / 2;
            if (mid * mid <= x) lo = mid; else hi = mid - 1;
        }
        cout << lo << "\n";
    }
}
```

INPUT

```
0
8
16
1000000000000
```

OUTPUT

```
0
2
4
1000000
```

**How it works:** `sqrt()` works on doubles and can be off by one for big numbers. Binary search is exact.



## 6 Monotonic stack

### Spot it when...

- next greater / next smaller element
- how many days until a warmer day
- stock span, histogram, "previous smaller"
- sum of minimums over all subarrays

### The idea

Keep a stack whose values are always increasing (or decreasing). When a new element breaks the order, pop elements; the new element is the "answer" for each one popped. Every element is pushed and popped once:  $O(n)$ .

### EXAMPLE Daily temperatures

For each day, how many days until a warmer temperature? (0 if never.)

```
int main() {
    int n; cin >> n;
    vector<int> t(n), ans(n, 0);
    for (auto &x : t) cin >> x;
    stack<int> st; // indices of days still waiting
    for (int i = 0; i < n; i++) {
        while (!st.empty() && t[st.top()] < t[i]) {
            ans[st.top()] = i - st.top(); // day i is the first warmer day
            st.pop();
        }
        st.push(i);
    }
    for (int x : ans) cout << x << " ";
    cout << "\n";
}
```

#### INPUT

```
8
73 74 75 71 69 72 76 73
```

#### OUTPUT

```
1 1 4 2 1 1 0 0
```

Time:  $O(n)$

### EXAMPLE Stock span

The span on day  $i$  is how many consecutive days up to and including today had a price  $\leq$  today's.

```
int main() {
    int n; cin >> n;
    vector<int> p(n);
    for (auto &x : p) cin >> x;
    stack<int> st; // indices with prices greater than later ones
    for (int i = 0; i < n; i++) {
        while (!st.empty() && p[st.top()] <= p[i]) st.pop();
        int span = st.empty() ? i + 1 : i - st.top();
        cout << span << " ";
        st.push(i);
    }
    cout << "\n";
}
```

INPUT

```
7
100 80 60 70 60 75 85
```

OUTPUT

```
1 1 1 2 1 4 6
```

**How it works:** after popping all smaller-or-equal prices, the top of the stack is the previous *greater* price. The span is the distance to it.

### EXAMPLE Largest rectangle in a histogram

Bars of width 1 have the given heights. Find the area of the largest rectangle inside the histogram.

```
int main() {
    int n; cin >> n;
    vector<long long> h(n);
    for (auto &x : h) cin >> x;
    h.push_back(0); // a 0 at the end flushes the stack
    stack<int> st;
    long long best = 0;
    for (int i = 0; i <= n; i++) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            long long height = h[st.top()]; st.pop();
            int leftEdge = st.empty() ? -1 : st.top(); // previous smaller bar
            best = max(best, height * (i - leftEdge - 1)); // i = next smaller bar
        }
        st.push(i);
    }
    cout << best << "\n";
}
```

INPUT

```
6
2 1 5 6 2 3
```

OUTPUT

```
10
```

**How it works:** when a bar is popped, we know both the first smaller bar to its right (i) and to its left (new top), so we know how wide its rectangle can be.

**Time:**  $O(n)$

## 7 Greedy

### Spot it when...

- pick a maximum number of non-overlapping things
- minimum resources at the same time (rooms, platforms)
- "can you reach...", minimum jumps or stops
- it feels like the locally best choice is always safe

### The idea

Make the best-looking choice at each step and never undo it. It usually needs sorting first. Greedy is fast but easy to get wrong: if you can find a small counter-example, switch to DP.

**EXAMPLE Jump game II: minimum jumps to the end**

nums[i] is the longest jump from index i. Print the minimum jumps to reach the last index, or -1 if impossible.

```
int minJumps(vector<int>& a) {
    int n = a.size(), jumps = 0, curEnd = 0, farthest = 0;
    for (int i = 0; i < n - 1; i++) {
        if (i > farthest) return -1;          // can't even reach i
        farthest = max(farthest, i + a[i]);
        if (i == curEnd) {                    // end of this jump's range
            if (farthest <= i) return -1;
            jumps++;
            curEnd = farthest;
        }
    }
    return jumps;
}

int main() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        vector<int> a(n);
        for (auto &x : a) cin >> x;
        cout << minJumps(a) << "\n";
    }
}
```

**INPUT**

```
2
5
2 3 1 1 4
5
3 2 1 0 4
```

**OUTPUT**

```
2
-1
```

**How it works:** think in layers: everything reachable with 1 jump, then with 2 jumps, and so on. `curEnd` is where the current layer ends.

### EXAMPLE Minimum railway platforms

Given arrival and departure times, find the fewest platforms so no train waits.

```
int main() {
    int n; cin >> n;
    vector<int> arr(n), dep(n);
    for (auto &x : arr) cin >> x;
    for (auto &x : dep) cin >> x;
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());
    int i = 0, j = 0, cur = 0, best = 0;
    while (i < n) {
        if (arr[i] <= dep[j]) { cur++; i++; } // a train arrives before the next leaves
        else { cur--; j++; } // a train leaves
        best = max(best, cur);
    }
    cout << best << "\n";
}
```

INPUT

```
6
900 940 950 1100 1500 1800
910 1200 1120 1130 1900 2000
```

OUTPUT

```
3
```

**How it works:** only the order of events matters, not which train is which. Sort arrivals and departures separately and sweep through time. (Same idea: "meeting rooms II".)

**Time:**  $O(n \log n)$

### EXAMPLE Gas station

Stations in a circle: gas[i] litres available, cost[i] to drive to the next. Find the starting station to go all the way round, or -1.

```
int main() {
    int n; cin >> n;
    vector<int> gas(n), cost(n);
    for (auto &x : gas) cin >> x;
    for (auto &x : cost) cin >> x;
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < n; i++) {
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) { start = i + 1; tank = 0; } // can't start anywhere before i+1
    }
    cout << (total >= 0 ? start : -1) << "\n";
}
```

INPUT

```
5
1 2 3 4 5
3 4 5 1 2
```

OUTPUT

```
3
```

**How it works:** if you run out of fuel at station i, no station between the old start and i could work either, so jump the start to i+1. If total gas  $\geq$  total cost, a solution exists.

### EXAMPLE Fractional knapsack

You can take fractions of items. Maximise value within weight W.

```
int main() {
    int n; double W; cin >> n >> W;
    vector<pair<double,double>> items(n);           // {value, weight}
    for (auto &[v, w] : items) cin >> v >> w;
    sort(items.begin(), items.end(), [](auto &a, auto &b) {
        return a.first / a.second > b.first / b.second; // best value per kg first
    });
    double total = 0;
    for (auto &[v, w] : items) {
        if (W <= 0) break;
        double take = min(w, W);
        total += v * (take / w);
        W -= take;
    }
    cout << fixed << setprecision(2) << total << "\n";
}
```

#### INPUT

```
3 50
60 10
100 20
120 30
```

#### OUTPUT

```
240.00
```

**How it works:** with fractions allowed, greedy by value-per-weight is optimal. For the 0/1 version (whole items only) greedy fails; use DP (§13).

## 8 Recursion & backtracking

### Spot it when...

- generate all subsets / permutations / combinations
- place items under rules (N-Queens, Sudoku)
- n is small ( $\leq 15-20$ )
- "print all ways"

### The idea

Build the answer one choice at a time. After exploring a choice, **undo it** (that's the "back" in backtracking) and try the next choice. Skip choices early when they can't lead to a valid answer (pruning).

```
void solve(state) {
    if (state is complete) { record answer; return; }
    for (each choice) {
        if (choice is not allowed) continue; // pruning
        make the choice;
        solve(next state);
        undo the choice;                      // backtrack
    }
}
```

**EXAMPLE All subsets (pick or skip)**

Print every subset of the numbers.

```
vector<int> a, cur;
void rec(int i) {
    if (i == (int)a.size()) {
        cout << "[";
        for (int k = 0; k < (int)cur.size(); k++) cout << (k ? " " : "") << cur[k];
        cout << "]" ";
        return;
    }
    rec(i + 1); // choice 1: skip a[i]
    cur.push_back(a[i]); // choice 2: take a[i]
    rec(i + 1);
    cur.pop_back(); // undo
}
int main() {
    int n; cin >> n; a.resize(n);
    for (auto &x : a) cin >> x;
    rec(0);
    cout << "\n";
}
```

**INPUT**

```
3
1 2 3
```

**OUTPUT**

```
[ ] [3] [2] [2 3] [1] [1 3] [1 2] [1 2 3]
```

**How it works:** every element has two choices, so there are  $2^n$  subsets.

**Time:**  $O(2^n \cdot n)$



**EXAMPLE All permutations**

Print all orderings of the numbers in increasing (dictionary) order.

```
int n;
vector<int> a, cur;
vector<bool> used;
void rec() {
    if ((int)cur.size() == n) {
        for (int x : cur) cout << x;
        cout << " ";
        return;
    }
    for (int i = 0; i < n; i++) {
        if (used[i]) continue;
        used[i] = true; cur.push_back(a[i]);
        rec();
        used[i] = false; cur.pop_back();    // undo
    }
}
int main() {
    cin >> n; a.resize(n); used.assign(n, false);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    rec();
    cout << "\n";
}
```

**INPUT**

```
3
3 1 2
```

**OUTPUT**

```
123 132 213 231 312 321
```

**Time:**  $O(n! \cdot n)$

### EXAMPLE Combination sum

Print every combination of candidates (each can be reused) that adds up to the target.

```
vector<int> c, cur;
void rec(int start, int remain) {
    if (remain == 0) {
        cout << "[";
        for (int k = 0; k < (int)cur.size(); k++) cout << (k ? " " : "") << cur[k];
        cout << "]\n";
        return;
    }
    for (int i = start; i < (int)c.size(); i++) {
        if (c[i] > remain) break;           // sorted, so later ones are too big as well
        cur.push_back(c[i]);
        rec(i, remain - c[i]);             // i, not i+1: the same number may be reused
        cur.pop_back();
    }
}
int main() {
    int n, target; cin >> n >> target;
    c.resize(n);
    for (auto &x : c) cin >> x;
    sort(c.begin(), c.end());
    rec(0, target);
}
```

#### INPUT

```
4 7
2 3 6 7
```

#### OUTPUT

```
[2 2 3]
[7]
```

**How it works:** starting the loop at `start` stops us from printing the same combination in a different order. Sorting lets us stop early with `break`.

### EXAMPLE N-Queens: count the solutions

Place  $n$  queens on an  $n \times n$  board so that no two attack each other. Count the ways.

```
int n, cnt;
vector<bool> col, d1, d2;           // used columns and diagonals
void rec(int r) {
    if (r == n) { cnt++; return; }
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r + c] || d2[r - c + n - 1]) continue; // attacked
        col[c] = d1[r + c] = d2[r - c + n - 1] = true;
        rec(r + 1);
        col[c] = d1[r + c] = d2[r - c + n - 1] = false;
    }
}
int main() {
    while (cin >> n) {
        cnt = 0;
        col.assign(n, false); d1.assign(2 * n, false); d2.assign(2 * n, false);
        rec(0);
        cout << n << " -> " << cnt << "\n";
    }
}
```

INPUT

```
4
8
```

OUTPUT

```
4 -> 2
8 -> 92
```

**How it works:** place one queen per row. On a diagonal, either  $r + c$  or  $r - c$  stays the same, so one array per diagonal direction tells us in  $O(1)$  whether a square is attacked.

## 9 Linked lists (fast & slow pointers)

### Spot it when...

- reverse a list, or part of it
- find the middle
- detect a cycle and where it starts
- merge sorted lists, remove the  $n$ -th node from the end

### The idea

Most linked-list tricks use two pointers. A **slow** pointer moves 1 step and a **fast** pointer moves 2 steps. A **dummy** head node avoids special cases when the head itself changes.

All examples in this section use this helper code (shown once):

```

struct Node {
    int val; Node* next;
    Node(int v) : val(v), next(nullptr) {}
};

Node* readList() { // reads n, then n values
    int n; cin >> n;
    Node dummy(0); Node* tail = &dummy;
    for (int i = 0; i < n; i++) { int x; cin >> x; tail->next = new Node(x); tail = tail->next; }
    return dummy.next;
}

void printList(Node* h) {
    for (; h; h = h->next) cout << h->val << " ";
    cout << "\n";
}

```

### EXAMPLE Reverse a linked list

Reverse the list by changing the links, without creating new nodes.

```

Node* reverseList(Node* head) {
    Node* prev = nullptr;
    while (head) {
        Node* nxt = head->next; // remember the rest
        head->next = prev; // flip the link
        prev = head;
        head = nxt;
    }
    return prev; // the old tail is the new head
}

int main() { printList(reverseList(readList())); }

```

#### INPUT

```

5
1 2 3 4 5

```

#### OUTPUT

```

5 4 3 2 1

```

**How it works:** walk once through the list, pointing each node back at the previous one. Three pointers: prev, current, next.

**Time:**  $O(n)$ ,  $O(1)$  memory

**EXAMPLE Middle of a linked list**

Print the middle value (for an even length, the second of the two middles).

```
int main() {
    for (int t = 0; t < 2; t++) {
        Node* head = readList();
        Node *slow = head, *fast = head;
        while (fast && fast->next) {
            slow = slow->next;          // 1 step
            fast = fast->next->next;     // 2 steps
        }
        cout << slow->val << "\n";
    }
}
```

**INPUT**

```
5
1 2 3 4 5
6
1 2 3 4 5 6
```

**OUTPUT**

```
3
4
```

**How it works:** when fast reaches the end it has walked twice as far as slow, so slow is in the middle.

**EXAMPLE Detect a cycle and find where it starts (Floyd)**

The last node links back to the node at index pos (-1 means no cycle). Find where the cycle starts.

```
int main() {
    int n; cin >> n;
    vector<Node*> nodes(n);
    for (int i = 0; i < n; i++) { int x; cin >> x; nodes[i] = new Node(x); }
    for (int i = 0; i + 1 < n; i++) nodes[i]->next = nodes[i + 1];
    int pos; cin >> pos;
    if (pos >= 0) nodes[n - 1]->next = nodes[pos];          // make the cycle
    Node *slow = nodes[0], *fast = nodes[0];
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {                                // they met: cycle exists
            Node* p = nodes[0];
            while (p != slow) { p = p->next; slow = slow->next; }
            cout << "cycle starts at value " << p->val << "\n";
            return 0;
        }
    }
    cout << "no cycle\n";
}
```

**INPUT**

```
4
3 2 0 -4
1
```

**OUTPUT**

```
cycle starts at value 2
```

**How it works:** if there's a cycle, fast eventually laps slow. After they meet, a pointer from the head and a pointer from the meeting point, both moving 1 step, meet exactly at the cycle's start.

**Time:**  $O(n)$ ,  $O(1)$  memory

### EXAMPLE Merge two sorted lists

Merge two sorted lists into one sorted list.

```
Node* mergeLists(Node* a, Node* b) {
    Node dummy(0); Node* tail = &dummy;    // dummy avoids a special case for the head
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;                  // attach whatever is left
    return dummy.next;
}

int main() {
    Node* a = readList();
    Node* b = readList();
    printList(mergeLists(a, b));
}
```

INPUT

```
3
1 2 4
3
1 3 4
```

OUTPUT

```
1 1 2 3 4 4
```

Time:  $O(n + m)$

## 10 Trees

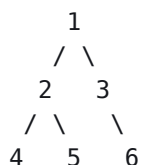
### Spot it when...

- depth, height, diameter, path sums
- traversal orders, level by level
- lowest common ancestor
- BST: sorted order, search, k-th smallest

### The idea

Almost every tree problem is a short recursive function: **solve the left subtree, solve the right subtree, combine**. Ask: "if I knew the answer for both children, how would I get the answer for this node?" Level-by-level problems use BFS with a queue.

All tree examples use this helper. It builds a tree from level-order input where  $-1$  means "no node". The sample tree is:



```

struct TreeNode {
    int val; TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

TreeNode* readTree() { // n, then level-order values, -1 = empty
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    if (n == 0 || a[0] == -1) return nullptr;
    TreeNode* root = new TreeNode(a[0]);
    queue<TreeNode*> q; q.push(root);
    int i = 1;
    while (!q.empty() && i < n) {
        TreeNode* cur = q.front(); q.pop();
        if (i < n && a[i] != -1) { cur->left = new TreeNode(a[i]); q.push(cur->left); }
        i++;
        if (i < n && a[i] != -1) { cur->right = new TreeNode(a[i]); q.push(cur->right); }
        i++;
    }
    return root;
}

```



### EXAMPLE The four traversals

Print preorder, inorder, postorder, and the tree level by level.

```
void pre(TreeNode* r) { if (!r) return; cout << r->val << " "; pre(r->left); pre(r->right); }
void in(TreeNode* r)  { if (!r) return; in(r->left); cout << r->val << " "; in(r->right); }
void post(TreeNode* r) { if (!r) return; post(r->left); post(r->right); cout << r->val << " "; }
int main() {
    TreeNode* root = readTree();
    cout << "pre: "; pre(root); cout << "\n";
    cout << "in:  "; in(root);  cout << "\n";
    cout << "post: "; post(root); cout << "\n";
    queue<TreeNode*> q; q.push(root);
    int level = 0;
    while (!q.empty()) {
        int sz = q.size(); // number of nodes on this level
        cout << "level " << level++ << ":";
        while (sz-->0) {
            TreeNode* cur = q.front(); q.pop();
            cout << " " << cur->val;
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        cout << "\n";
    }
}
```

#### INPUT

```
7
1 2 3 4 5 -1 6
```

#### OUTPUT

```
pre: 1 2 4 5 3 6
in:   4 2 5 1 3 6
post: 4 5 2 6 3 1
level 0: 1
level 1: 2 3
level 2: 4 5 6
```

**How it works:** pre = node first, in = node in the middle, post = node last. For levels, read `q.size()` before the loop so you know where one level ends.

### EXAMPLE Height and diameter

Height = number of nodes on the longest root-to-leaf path. Diameter = number of edges on the longest path between any two nodes.

```
int diameter = 0;
int height(TreeNode* r) {
    if (!r) return 0;
    int lh = height(r->left), rh = height(r->right);
    diameter = max(diameter, lh + rh);    // longest path that bends at r
    return 1 + max(lh, rh);
}
int main() {
    TreeNode* root = readTree();
    int h = height(root);
    cout << "height " << h << ", diameter " << diameter << "\n";
}
```

INPUT

```
7
1 2 3 4 5 -1 6
```

OUTPUT

```
height 3, diameter 4
```

**How it works:** the longest path through a node uses the deepest branch on each side, so it's left height + right height. One DFS computes heights and updates the best diameter on the way.

Time:  $O(n)$

### EXAMPLE Left view and right view

Print the first node and the last node of every level.

```
int main() {
    TreeNode* root = readTree();
    vector<int> leftV, rightV;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front(); q.pop();
            if (i == 0) leftV.push_back(cur->val);
            if (i == sz - 1) rightV.push_back(cur->val);
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
    }
    cout << "left:"; for (int x : leftV) cout << " " << x;
    cout << "\nright:"; for (int x : rightV) cout << " " << x;
    cout << "\n";
}
```

INPUT

```
7
1 2 3 4 5 -1 6
```

OUTPUT

```
left: 1 2 4
right: 1 3 6
```

**EXAMPLE Lowest common ancestor**

For each pair of values, print their lowest common ancestor (the deepest node that has both below it).

```
TreeNode* lca(TreeNode* r, int a, int b) {
    if (!r || r->val == a || r->val == b) return r;
    TreeNode* L = lca(r->left, a, b);
    TreeNode* R = lca(r->right, a, b);
    if (L && R) return r;           // one on each side: r is the meeting point
    return L ? L : R;
}

int main() {
    TreeNode* root = readTree();
    int q; cin >> q;
    while (q--) { int a, b; cin >> a >> b; cout << lca(root, a, b)->val << "\n"; }
}
```

**INPUT**

```
7
1 2 3 4 5 -1 6
3
4 5
4 6
5 2
```

**OUTPUT**

```
2
1
2
```

**How it works:** each call returns a found node (or null). The first node where both sides return something is the LCA. If a node is itself one of the targets, it's returned straight away.

**Time:**  $O(n)$  per query

**EXAMPLE Binary search tree: insert, search, k-th smallest, validate**

Build a BST by inserting values, then answer a few questions about it.

```
TreeNode* insert(TreeNode* r, int v) {
    if (!r) return new TreeNode(v);
    if (v < r->val) r->left = insert(r->left, v);
    else r->right = insert(r->right, v);
    return r;
}

bool search(TreeNode* r, int v) {
    while (r) { if (v == r->val) return true; r = v < r->val ? r->left : r->right; }
    return false;
}

void inorder(TreeNode* r, vector<int>& out) {
    if (!r) return; inorder(r->left, out); out.push_back(r->val); inorder(r->right, out);
}

bool valid(TreeNode* r, long long lo, long long hi) {    // every value must be in (lo, hi)
    if (!r) return true;
    if (r->val <= lo || r->val >= hi) return false;
    return valid(r->left, lo, r->val) && valid(r->right, r->val, hi);
}

int main() {
    int n; cin >> n;
    TreeNode* root = nullptr;
    for (int i = 0; i < n; i++) { int v; cin >> v; root = insert(root, v); }
    vector<int> s; inorder(root, s);
    cout << "inorder: "; for (int x : s) cout << " " << x; cout << "\n";
    cout << "search 60: " << (search(root, 60) ? "found" : "not found") << "\n";
    cout << "search 65: " << (search(root, 65) ? "found" : "not found") << "\n";
    cout << "3rd smallest: " << s[2] << "\n";
    cout << "valid BST: " << (valid(root, LLONG_MIN, LLONG_MAX) ? "yes" : "no") << "\n";
}
```

**INPUT**

```
7
50 30 70 20 40 60 80
```

**OUTPUT**

```
inorder: 20 30 40 50 60 70 80
search 60: found
search 65: not found
3rd smallest: 40
valid BST: yes
```

**How it works:** the inorder traversal of a BST is always sorted, so the k-th smallest is the k-th value in inorder. To validate, pass down the allowed (low, high) range; checking only a node against its children is a classic mistake.

## 11 Graphs

### Spot it when...

- grids (each cell is a node)
- cities and roads, networks, friendships
- tasks with dependencies ("A must happen before B")
- fewest steps → BFS; weighted shortest path → Dijkstra

### The idea

Store the graph as an adjacency list: `vector<vector<int>>` `adj(n)`. Use **BFS** (queue) for shortest paths when every edge costs the same, **DFS** (recursion) to explore everything reachable, **Dijkstra** for weighted edges, and **topological sort** for dependencies.

| Algorithm               | Use it for                                                      | Time                |
|-------------------------|-----------------------------------------------------------------|---------------------|
| BFS                     | shortest path in unweighted graphs, levels, multi-source spread | $O(V + E)$          |
| DFS                     | connected components, flood fill, cycle detection               | $O(V + E)$          |
| Topological sort (Kahn) | ordering with dependencies, cycle check in directed graphs      | $O(V + E)$          |
| Dijkstra                | shortest path with non-negative weights                         | $O((V + E) \log V)$ |
| Bellman–Ford            | negative weights, detecting negative cycles                     | $O(V \cdot E)$      |
| Floyd–Warshall          | all-pairs shortest paths, $V \leq$ about 400                    | $O(V^3)$            |
| Kruskal / Prim          | minimum spanning tree                                           | $O(E \log E)$       |

**EXAMPLE BFS: shortest path in an unweighted graph**

Find the distance from node 0 to every node, and print the actual path to the last node.

```
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v); adj[v].push_back(u);    // undirected
    }
    vector<int> dist(n, -1), parent(n, -1);
    queue<int> q;
    dist[0] = 0; q.push(0);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) { dist[v] = dist[u] + 1; parent[v] = u; q.push(v); }
    }
    cout << "dist:"; for (int d : dist) cout << " " << d; cout << "\n";
    vector<int> path;
    for (int v = n - 1; v != -1; v = parent[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    cout << "path to " << n - 1 << ":"; for (int v : path) cout << " " << v; cout << "\n";
}
```

**INPUT**

```
6 6
0 1
0 2
1 3
2 3
3 4
4 5
```

**OUTPUT**

```
dist: 0 1 1 2 3 4
path to 5: 0 1 3 4 5
```

**How it works:** BFS visits nodes in order of distance, so the first time we reach a node is the shortest way there. Saving each node's parent lets us walk the path backwards.

**Time:**  $O(V + E)$

**EXAMPLE DFS: connected components**

Count the groups of connected nodes and list each group.

```
vector<vector<int>> adj;
vector<bool> seen;
void dfs(int u, vector<int>& group) {
    seen[u] = true; group.push_back(u);
    for (int v : adj[u]) if (!seen[v]) dfs(v, group);
}
int main() {
    int n, m; cin >> n >> m;
    adj.assign(n, {}); seen.assign(n, false);
    for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; adj[u].push_back(v);
    adj[v].push_back(u); }
    int count = 0;
    for (int i = 0; i < n; i++)
        if (!seen[i]) {
            vector<int> g; dfs(i, g); count++;
            cout << "{"; for (int k = 0; k < (int)g.size(); k++) cout << (k ? " " : "") << g[k];
            cout << "} ";
        }
    cout << "\ncomponents: " << count << "\n";
}
```

**INPUT**

```
7 3
0 1
1 2
3 4
```

**OUTPUT**

```
{0 1 2} {3 4} {5} {6}
components: 4
```

**How it works:** start a DFS from every node that hasn't been seen yet. Each new start is a new component.

**EXAMPLE** Number of islands (grid DFS / flood fill)

1 = land, 0 = water. Count the islands (land cells connected up, down, left or right).

```
int n, m;
vector<string> g;
void fill(int r, int c) {
    if (r < 0 || r >= n || c < 0 || c >= m || g[r][c] != '1') return;
    g[r][c] = '0'; // mark as visited by sinking it
    fill(r + 1, c); fill(r - 1, c); fill(r, c + 1); fill(r, c - 1);
}
int main() {
    cin >> n >> m; g.resize(n);
    for (auto &row : g) cin >> row;
    int islands = 0;
    for (int r = 0; r < n; r++)
        for (int c = 0; c < m; c++)
            if (g[r][c] == '1') { islands++; fill(r, c); }
    cout << islands << "\n";
}
```

**INPUT**

```
4 5
11000
11000
00100
00011
```

**OUTPUT**

```
3
```

**How it works:** each cell is a node with up to 4 neighbours. Every time we find unvisited land, we've found a new island and flood it so we don't count it again.

**Time:**  $O(n \cdot m)$



**EXAMPLE Rotting oranges (multi-source BFS)**

2 = rotten, 1 = fresh, 0 = empty. Each minute, rot spreads to neighbouring fresh oranges. How many minutes until none are fresh? (-1 if impossible.)

```
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> g(n, vector<int>(m));
    queue<pair<int,int>> q;
    int fresh = 0;
    for (int r = 0; r < n; r++)
        for (int c = 0; c < m; c++) {
            cin >> g[r][c];
            if (g[r][c] == 2) q.push({r, c});           // ALL rotten oranges start together
            if (g[r][c] == 1) fresh++;
        }
    int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1}, minutes = 0;
    while (!q.empty() && fresh > 0) {
        int sz = q.size();
        while (sz--) {
            auto [r, c] = q.front(); q.pop();
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nr >= n || nc < 0 || nc >= m || g[nr][nc] != 1) continue;
                g[nr][nc] = 2; fresh--; q.push({nr, nc});
            }
        }
        minutes++;                                     // one BFS level = one minute
    }
    cout << (fresh == 0 ? minutes : -1) << "\n";
}
```

**INPUT**

```
3 3
2 1 1
1 1 0
0 1 1
```

**OUTPUT**

```
4
```

**How it works:** put every starting point in the queue before BFS begins. The BFS levels then measure time from the nearest source.

**EXAMPLE Topological sort (Kahn's algorithm) and cycle check**

Order the tasks so every edge  $u \rightarrow v$  has  $u$  before  $v$ . If there's a cycle, no order exists.

```
int main() {
    int t; cin >> t;
    while (t--) {
        int n, m; cin >> n >> m;
        vector<vector<int>> adj(n);
        vector<int> indeg(n, 0);
        for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; adj[u].push_back(v); indeg[v]++; }

        priority_queue<int, vector<int>, greater<int>> q; // min-heap: smallest label first
        for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
        vector<int> order;
        while (!q.empty()) {
            int u = q.top(); q.pop();
            order.push_back(u);
            for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
        }
        if ((int)order.size() < n) cout << "cycle, no order\n";
        else { for (int x : order) cout << x << " "; cout << "\n"; }
    }
}
```

**INPUT**

```
2
6 6
5 2
5 0
4 0
4 1
2 3
3 1
3 3
0 1
1 2
2 0
```

**OUTPUT**

```
4 5 0 2 3 1
cycle, no order
```

**How it works:** repeatedly take a task with no remaining prerequisites (in-degree 0). If some tasks never reach in-degree 0, they're stuck in a cycle. A plain `queue` also works; the min-heap just gives the smallest valid order.

**Time:**  $O((V + E) \log V)$  here,  $O(V + E)$  with a queue

**EXAMPLE Bipartite check (two-colouring)**

Can the nodes be coloured with 2 colours so that every edge joins different colours?

```
int main() {
    int t; cin >> t;
    while (t--) {
        int n, m; cin >> n >> m;
        vector<vector<int>> adj(n);
        for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; adj[u].push_back(v);
adj[v].push_back(u); }
        vector<int> color(n, -1);
        bool ok = true;
        for (int s = 0; s < n && ok; s++) {
            if (color[s] != -1) continue;
            queue<int> q; q.push(s); color[s] = 0;
            while (!q.empty() && ok) {
                int u = q.front(); q.pop();
                for (int v : adj[u]) {
                    if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
                    else if (color[v] == color[u]) ok = false;    // same colour on both ends
                }
            }
        }
        cout << (ok ? "YES" : "NO") << "\n";
    }
}
```

**INPUT**

```
2
4 4
0 1
1 2
2 3
3 0
3 3
0 1
1 2
2 0
```

**OUTPUT**

```
YES
NO
```

**How it works:** colour neighbours opposite colours as you BFS. A conflict means an odd-length cycle, so it's not bipartite (the square works, the triangle doesn't).

**EXAMPLE Dijkstra: weighted shortest paths**

Directed edges  $u \rightarrow v$  with weight  $w$ . Print the shortest distance from node 0 to every node.

```
int main() {
    int n, m; cin >> n >> m;
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) { int u, v, w; cin >> u >> v >> w; adj[u].push_back({v, w}); }
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[0] = 0; pq.push({0, 0});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // outdated entry
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    for (auto d : dist) cout << (d == LLONG_MAX ? -1 : d) << " ";
    cout << "\n";
}
```

**INPUT**

```
5 6
0 1 4
0 2 1
2 1 2
1 3 1
2 3 5
3 4 3
```

**OUTPUT**

```
0 3 1 4 7
```

**How it works:** always expand the closest unfinished node. Going  $0 \rightarrow 2 \rightarrow 1$  (cost 3) beats the direct edge  $0 \rightarrow 1$  (cost 4). Dijkstra fails with negative weights; use Bellman–Ford then.

**Time:**  $O((V + E) \log V)$

### EXAMPLE Floyd–Warshall: all-pairs shortest paths

Print the shortest distance between every pair of nodes.

```
int main() {
    int n, m; cin >> n >> m;
    const long long INF = 1e18;
    vector<vector<long long>> d(n, vector<long long>(n, INF));
    for (int i = 0; i < n; i++) d[i][i] = 0;
    for (int i = 0; i < m; i++) { int u, v; long long w; cin >> u >> v >> w; d[u][v] = min(d[u][v], w); }

    for (int k = 0; k < n; k++) // k MUST be the outer loop
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    for (auto &row : d) {
        for (auto x : row) cout << (x >= INF ? -1 : x) << " ";
        cout << "\n";
    }
}
```

#### INPUT

```
4 7
0 1 3
0 3 7
1 0 8
1 2 2
2 0 5
2 3 1
3 0 2
```

#### OUTPUT

```
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```

**How it works:** after round  $k$ ,  $d[i][j]$  is the best path that only uses nodes  $0..k$  in the middle. Putting  $k$  in an inner loop is a common bug.

**Time:**  $O(n^3)$

## 12 Union-find (Disjoint Set Union)

### Spot it when...

- groups that keep merging
- "are  $a$  and  $b$  connected?" asked many times
- detect a cycle in an undirected graph
- minimum spanning tree (Kruskal)

### The idea

Every group has a leader. `find(x)` returns  $x$ 's leader; `unite(a, b)` merges two groups. With path compression and union by size, both are almost  $O(1)$ .

```

struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};

```

#### EXAMPLE Redundant edge and number of groups

Add edges one by one. Print the first edge that joins two nodes already connected (it makes a cycle), and the final number of groups.

```

int main() {
    int n, m; cin >> n >> m;
    DSU d(n);
    int groups = n;
    bool printed = false;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        if (!d.unite(u, v)) {
            if (!printed) { cout << "redundant: " << u << " " << v << "\n"; printed = true; }
            else groups--;
        }
    }
    cout << "groups: " << groups << "\n";
}

```

#### INPUT

```

5 4
0 1
1 2
2 0
3 4

```

#### OUTPUT

```

redundant: 2 0
groups: 2

```

**How it works:** `unite` returns false when both ends already share a leader, which means this edge closes a cycle.

### EXAMPLE Kruskal's minimum spanning tree

Connect all nodes with the smallest total edge weight.

```
int main() {
    int n, m; cin >> n >> m;
    vector<array<int,3>> edges(m);           // {weight, u, v}
    for (auto &e : edges) cin >> e[1] >> e[2] >> e[0];
    sort(edges.begin(), edges.end());       // cheapest first
    DSU d(n);
    long long total = 0;
    for (auto &[w, u, v] : edges)
        if (d.unite(u, v)) { total += w; cout << u << " " << v << " " << w << "\n"; }
    cout << "total " << total << "\n";
}
```

#### INPUT

```
4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
```

#### OUTPUT

```
2 3 4
0 3 5
0 1 10
total 19
```

**How it works:** take edges from cheapest to most expensive and keep an edge only if it joins two different groups.

**Time:**  $O(E \log E)$

## 13 Dynamic programming

### Spot it when...

- count the number of ways
- minimum / maximum cost, longest / shortest
- each choice affects what you can do later
- brute-force recursion repeats the same sub-problems

### The idea

DP = recursion + remembering answers. Solve each small sub-problem once and reuse it. Every DP needs four things: a **state** (what  $dp[i]$  means), a **transition** (how  $dp[i]$  comes from smaller states), **base cases**, and the **final answer**.

#### Memoisation (top-down)

How write the recursion, cache results in an array

Pros easy to write from the brute force; only visits needed states

Cons deep recursion can crash for  $n \approx 10^6$

#### Tabulation (bottom-up)

fill a table in a loop from small to big

no recursion limit, often faster, easy to save memory

you must work out the right filling order

**How to find the state:** ask "what is the smallest information I need to describe where I am?" Usually an index ( $dp[i]$ ), two indices for two strings ( $dp[i][j]$ ), or index + remaining capacity ( $dp[i][w]$ ).

**EXAMPLE Climbing stairs**

You can climb 1 or 2 steps at a time. How many ways to reach step  $n$ ?

```
int main() {
    int n;
    while (cin >> n) {
        vector<long long> dp(n + 1);
        dp[0] = 1; dp[1] = 1;           // base cases
        for (int i = 2; i <= n; i++)
            dp[i] = dp[i - 1] + dp[i - 2]; // last move was 1 step or 2 steps
        cout << n << " -> " << dp[n] << "\n";
    }
}
```

**INPUT**

```
5
10
45
```

**OUTPUT**

```
5 -> 8
10 -> 89
45 -> 1836311903
```

**How it works:** state:  $dp[i]$  = ways to reach step  $i$ . To land on step  $i$ , you came from  $i-1$  or  $i-2$ .



**EXAMPLE** Frog jump: memoisation and tabulation side by side

A frog on stone 0 jumps 1 or 2 stones forward. A jump from  $i$  to  $j$  costs  $|h[i] - h[j]|$ . Find the minimum cost to reach the last stone.

```
vector<int> h;
vector<long long> memo;
long long solve(int i) {                // min cost to reach stone i
    if (i == 0) return 0;
    if (memo[i] != -1) return memo[i];  // already computed
    long long best = solve(i - 1) + abs(h[i] - h[i - 1]);
    if (i > 1) best = min(best, solve(i - 2) + abs(h[i] - h[i - 2]));
    return memo[i] = best;
}
int main() {
    int n; cin >> n; h.resize(n);
    for (auto &x : h) cin >> x;
    memo.assign(n, -1);
    cout << "memo: " << solve(n - 1) << "\n";

    vector<long long> dp(n, 0);          // same idea, filled in a loop
    for (int i = 1; i < n; i++) {
        dp[i] = dp[i - 1] + abs(h[i] - h[i - 1]);
        if (i > 1) dp[i] = min(dp[i], dp[i - 2] + abs(h[i] - h[i - 2]));
    }
    cout << "table: " << dp[n - 1] << "\n";
}
```

**INPUT**

```
4
10 20 30 10
```

**OUTPUT**

```
memo: 20
table: 20
```

**How it works:** both versions use the same state and transition. Memo starts at the answer and recurses down; the table starts at the base case and builds up.

### EXAMPLE House robber

Rob houses for money, but never two next to each other. Find the maximum total.

```
int main() {
    int n; cin >> n;
    long long take = 0, skip = 0;           // best total if we take / skip the current house
    for (int i = 0; i < n; i++) {
        long long x; cin >> x;
        long long newTake = skip + x;      // can only take if we skipped the previous one
        skip = max(skip, take);
        take = newTake;
    }
    cout << max(take, skip) << "\n";
}
```

INPUT

```
5
2 7 9 3 1
```

OUTPUT

```
12
```

**How it works:**  $dp[i] = \max(dp[i-1], dp[i-2] + a[i])$ . Since we only need the last two values, two variables replace the whole array ( $2 + 9 + 1 = 12$ ).

**Time:**  $O(n)$ ,  $O(1)$  memory

### EXAMPLE 0/1 knapsack

Each item has a weight and value and can be taken at most once. Maximise value with total weight  $\leq W$ .

```
int main() {
    int n, W; cin >> n >> W;
    vector<long long> dp(W + 1, 0);        // dp[w] = best value with capacity w
    for (int i = 0; i < n; i++) {
        int wt; long long val; cin >> wt >> val;
        for (int w = W; w >= wt; w--)      // go DOWN so each item is used once
            dp[w] = max(dp[w], dp[w - wt] + val);
    }
    cout << dp[W] << "\n";
}
```

INPUT

```
3 50
10 60
20 100
30 120
```

OUTPUT

```
220
```

**How it works:** for each item: skip it ( $dp[w]$  stays) or take it ( $dp[w - wt] + val$ ). Looping  $w$  downwards makes sure  $dp[w - wt]$  is still the value *before* this item. Looping upwards would allow reusing the item (that's the unbounded version).

**Time:**  $O(n \cdot W)$

**EXAMPLE Coin change: fewest coins, and number of ways**

With unlimited coins of each type, find (a) the fewest coins that make the amount and (b) the number of different combinations.

```
int main() {
    int n, amount; cin >> n >> amount;
    vector<int> coins(n);
    for (auto &x : coins) cin >> x;
    const int INF = 1e9;
    vector<int> fewest(amount + 1, INF);
    fewest[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a && fewest[a - c] != INF) fewest[a] = min(fewest[a], fewest[a - c] + 1);
    vector<long long> ways(amount + 1, 0);
    ways[0] = 1;
    for (int c : coins) // coins OUTER: counts combinations, not orders
        for (int a = c; a <= amount; a++) ways[a] += ways[a - c];
    cout << "fewest coins: " << (fewest[amount] == INF ? -1 : fewest[amount]) << "\n";
    cout << "ways: " << ways[amount] << "\n";
}
```

**INPUT**

```
3 11
1 2 5
```

**OUTPUT**

```
fewest coins: 3
ways: 11
```

**How it works:** fewest:  $11 = 5 + 5 + 1$ . For counting, putting coins in the outer loop means  $\{1,2\}$  and  $\{2,1\}$  are counted once. Swap the loops and you'd count orderings instead.

**EXAMPLE Longest common subsequence (and print it)**

Find the longest sequence of characters that appears in both strings in the same order (not necessarily together).

```
int main() {
    string a, b; cin >> a >> b;
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
                : max(dp[i-1][j], dp[i][j-1]);

    string s;
    // walk back to rebuild the answer
    for (int i = n, j = m; i > 0 && j > 0; ) {
        if (a[i-1] == b[j-1]) { s += a[i-1]; i--; j--; }
        else if (dp[i-1][j] >= dp[i][j-1]) i--;
        else j--;
    }
    reverse(s.begin(), s.end());
    cout << dp[n][m] << " " << s << "\n";
}
```

**INPUT**

abcde ace

**OUTPUT**

3 ace

**How it works:**  $dp[i][j]$  = LCS of the first  $i$  letters of  $a$  and first  $j$  letters of  $b$ . If the last letters match, use them; otherwise drop one letter from either string.

**Time:**  $O(n \cdot m)$

### EXAMPLE Edit distance

Minimum insertions, deletions and replacements to turn word a into word b.

```
int main() {
    string a, b;
    while (cin >> a >> b) {
        int n = a.size(), m = b.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1));
        for (int i = 0; i <= n; i++) dp[i][0] = i;    // delete everything
        for (int j = 0; j <= m; j++) dp[0][j] = j;    // insert everything
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
                else dp[i][j] = 1 + min({dp[i-1][j],        // delete
                                         dp[i][j-1],        // insert
                                         dp[i-1][j-1]});    // replace

        cout << a << " -> " << b << ": " << dp[n][m] << "\n";
    }
}
```

#### INPUT

```
horse ros
intention execution
```

#### OUTPUT

```
horse -> ros: 3
intention -> execution: 5
```

Time:  $O(n \cdot m)$

### EXAMPLE Minimum path sum in a grid

Move only right or down from the top-left to the bottom-right. Minimise the sum of the cells you pass.

```
int main() {
    int n, m; cin >> n >> m;
    vector<vector<long long>> g(n, vector<long long>(m));
    for (auto &row : g) for (auto &x : row) cin >> x;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (i == 0 && j == 0) continue;
            long long up = (i > 0) ? g[i-1][j] : LLONG_MAX;
            long long left = (j > 0) ? g[i][j-1] : LLONG_MAX;
            g[i][j] += min(up, left);    // reuse the grid as the dp table
        }
    cout << g[n-1][m-1] << "\n";
}
```

#### INPUT

```
3 3
1 3 1
1 5 1
4 2 1
```

#### OUTPUT

```
7
```

**How it works:** every cell can only be reached from above or from the left, so its best cost is its own value + the cheaper of those two. (Path:  $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$ .)

**EXAMPLE Longest increasing subsequence ( $O(n^2)$ , with the sequence)**

Find the length of the longest strictly increasing subsequence and print one.

```
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    vector<int> dp(n, 1), prev(n, -1);          // dp[i] = LIS ending at i
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) { dp[i] = dp[j] + 1; prev[i] = j; }
    int best = max_element(dp.begin(), dp.end()) - dp.begin();
    vector<int> seq;
    for (int i = best; i != -1; i = prev[i]) seq.push_back(a[i]);
    reverse(seq.begin(), seq.end());
    cout << dp[best] << "\n";
    for (int x : seq) cout << x << " ";
    cout << "\n";
}
```

**INPUT**

```
8
10 9 2 5 3 7 101 18
```

**OUTPUT**

```
4
2 5 7 101
```

**How it works:**  $dp[i]$  looks back at every smaller element before  $i$ . For  $n$  up to  $10^5$ , use the  $O(n \log n)$  `lower_bound` version from the STL notes.

**Time:**  $O(n^2)$

**EXAMPLE Partition into two equal-sum halves**

Can the numbers be split into two groups with equal sums?

```
int main() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        vector<int> a(n);
        int total = 0;
        for (auto &x : a) { cin >> x; total += x; }
        if (total % 2) { cout << "NO\n"; continue; } // odd total: impossible
        int half = total / 2;
        vector<bool> can(half + 1, false);
        can[0] = true;
        for (int x : a)
            for (int s = half; s >= x; s--) // downwards: each number once
                if (can[s - x]) can[s] = true;
        cout << (can[half] ? "YES" : "NO") << "\n";
    }
}
```

**INPUT**

```
2
4
1 5 11 5
4
1 2 3 5
```

**OUTPUT**

```
YES
NO
```

**How it works:** this is subset-sum in disguise: we just need one group that sums to  $\text{total}/2$  ( $11 = 1 + 5 + 5$ ).

**EXAMPLE Matrix chain multiplication (interval DP)**

Matrix  $i$  has size  $d[i-1] \times d[i]$ . Find the cheapest order to multiply them all (fewest scalar multiplications).

```
int main() {
    int t; cin >> t;
    while (t--) {
        int k; cin >> k;
        vector<long long> d(k);
        for (auto &x : d) cin >> x;
        int n = k - 1; // number of matrices
        vector<vector<long long>> dp(n, vector<long long>(n, 0));
        for (int len = 2; len <= n; len++) // short chains first
            for (int i = 0; i + len - 1 < n; i++) {
                int j = i + len - 1;
                dp[i][j] = LLONG_MAX;
                for (int s = i; s < j; s++) // where to split the chain
                    dp[i][j] = min(dp[i][j], dp[i][s] + dp[s+1][j] + d[i] * d[s+1] * d[j+1]);
            }
        cout << dp[0][n - 1] << "\n";
    }
}
```

**INPUT**

```
2
4
10 30 5 60
5
40 20 30 10 30
```

**OUTPUT**

```
4500
26000
```

**How it works:** interval DP:  $dp[i][j]$  = best cost for matrices  $i..j$ . Try every split point and solve shorter intervals first. Burst Balloons and similar problems use the same shape.

**Time:**  $O(n^3)$



### EXAMPLE Travelling salesman (bitmask DP)

Visit every city exactly once starting and ending at city 0, with the lowest total distance ( $n \leq 15$  or so).

```
int main() {
    int n; cin >> n;
    vector<vector<int>> d(n, vector<int>(n));
    for (auto &row : d) for (auto &x : row) cin >> x;
    const int INF = 1e9;
    // dp[mask][i] = shortest path that visits the cities in mask and ends at i
    vector<vector<int>> dp(1 << n, vector<int>(n, INF));
    dp[1][0] = 0; // only city 0 visited
    for (int mask = 1; mask < (1 << n); mask++)
        for (int i = 0; i < n; i++) {
            if (dp[mask][i] == INF || !(mask & (1 << i))) continue;
            for (int j = 0; j < n; j++)
                if (!(mask & (1 << j)))
                    dp[mask | (1 << j)][j] = min(dp[mask | (1 << j)][j], dp[mask][i] + d[i][j]);
        }
    int best = INF;
    for (int i = 1; i < n; i++) best = min(best, dp[(1 << n) - 1][i] + d[i][0]);
    cout << best << "\n";
}
```

#### INPUT

```
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

#### OUTPUT

```
80
```

**How it works:** the mask records which cities are already visited; the second index is where we are now. That's  $2^n \cdot n$  states instead of  $n!$  orderings.

**Time:**  $O(2^n \cdot n^2)$

## 14 Heaps

### Spot it when...

- running median of a stream
- top K / K-th largest (see STL notes)
- always process the smallest / cheapest next
- merge K sorted things

### The idea

A heap gives the max (or min) in  $O(1)$  and inserts/removes in  $O(\log n)$ . The two-heaps trick keeps the smaller half in a max-heap and the bigger half in a min-heap, so the middle is always on top.

### EXAMPLE Median of a stream (two heaps)

Numbers arrive one by one. After each one, print the median so far.

```
int main() {
    int n; cin >> n;
    priority_queue<int> low; // max-heap: smaller half
    priority_queue<int, vector<int>, greater<int>> high; // min-heap: bigger half
    cout << fixed << setprecision(1);
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        if (low.empty() || x <= low.top()) low.push(x); else high.push(x);
        // rebalance: low may have at most one extra element
        if (low.size() > high.size() + 1) { high.push(low.top()); low.pop(); }
        else if (high.size() > low.size()) { low.push(high.top()); high.pop(); }
        double med = (low.size() > high.size()) ? low.top() : (low.top() + (double)high.top()) /
2;
        cout << med << " ";
    }
    cout << "\n";
}
```

INPUT

```
4
5 15 1 3
```

OUTPUT

```
5.0 10.0 5.0 4.0
```

**How it works:** the top of the max-heap is the largest of the small half, and the top of the min-heap is the smallest of the big half. Those two are the middle values.

**Time:**  $O(\log n)$  per number

## 15 String algorithms

### Spot it when...

- find a pattern inside a long text, many times
- longest prefix that is also a suffix
- palindromic substrings
- compare substrings quickly

### The idea

**KMP** finds a pattern in  $O(n + m)$  using the prefix function (for each position, the length of the longest proper prefix that is also a suffix). For palindromes, **expand around each centre**.

**EXAMPLE KMP pattern search**

Print the prefix function of the pattern, and every index where the pattern occurs in the text.

```
vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int k = pi[i - 1];
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];    // fall back to a shorter border
        if (s[i] == s[k]) k++;
        pi[i] = k;
    }
    return pi;
}

int main() {
    string text, pat; cin >> text >> pat;
    vector<int> pi = prefixFunction(pat);
    cout << "prefix function:"; for (int x : pi) cout << " " << x; cout << "\n";
    vector<int> all = prefixFunction(pat + "#" + text);    // # never appears in the text
    cout << "found at:";
    for (int i = 0; i < (int)all.size(); i++)
        if (all[i] == (int)pat.size()) cout << " " << i - 2 * (int)pat.size();
    cout << "\n";
}
```

**INPUT**

ababcabcabababd ababd

**OUTPUT**

prefix function: 0 0 1 2 0  
found at: 10

**How it works:** joining pattern + # + text and computing the prefix function finds every full match: wherever the value equals the pattern length.

**Time:**  $O(n + m)$

### EXAMPLE Longest palindromic substring (expand around centre)

Find the longest substring that reads the same forwards and backwards.

```
int main() {
    string s;
    while (cin >> s) {
        int n = s.size(), start = 0, best = 1;
        for (int c = 0; c < n; c++)
            for (int odd = 0; odd < 2; odd++) {           // centre on a letter, or between two
                int l = c, r = c + odd;
                while (l >= 0 && r < n && s[l] == s[r]) { l--; r++; }
                int len = r - l - 1;
                if (len > best) { best = len; start = l + 1; }
            }
        cout << s << " -> " << s.substr(start, best) << "\n";
    }
}
```

#### INPUT

```
babad
cbbd
```

#### OUTPUT

```
babad -> bab
cbbd -> bb
```

**How it works:** every palindrome has a centre: a letter (odd length) or a gap between two letters (even length). Grow outwards from all  $2n - 1$  centres.

**Time:**  $O(n^2)$

## 16 Trie (prefix tree)

### Spot it when...

- many words, and questions about prefixes
- autocomplete, "how many words start with..."
- word search on a grid with a dictionary
- maximum XOR of two numbers (a trie over bits)

### The idea

A tree where each edge is a letter. Words with the same prefix share the same path, so a prefix lookup takes  $O(\text{length of the prefix})$ , no matter how many words there are.

**EXAMPLE Insert, search, and count words with a prefix**

Insert words, then answer: is this exact word stored? How many stored words start with this prefix?

```
struct Trie {
    struct Node { int child[26]; int pass = 0; bool end = false; Node() { memset(child, -1,
sizeof child); } };
    vector<Node> t{Node()}; // node 0 is the root
    void insert(const string& w) {
        int cur = 0;
        for (char ch : w) {
            int c = ch - 'a';
            if (t[cur].child[c] == -1) { t[cur].child[c] = t.size(); t.push_back(Node()); }
            cur = t[cur].child[c];
            t[cur].pass++; // one more word passes through here
        }
        t[cur].end = true;
    }
    int walk(const string& w) { // node for this prefix, or -1
        int cur = 0;
        for (char ch : w) { cur = t[cur].child[ch - 'a']; if (cur == -1) return -1; }
        return cur;
    }
    bool search(const string& w) { int n = walk(w); return n != -1 && t[n].end; }
    int countPrefix(const string& p) { int n = walk(p); return n == -1 ? 0 : t[n].pass; }
};

int main() {
    int n; cin >> n;
    Trie tr;
    for (int i = 0; i < n; i++) { string w; cin >> w; tr.insert(w); }
    int q; cin >> q;
    while (q--) {
        string type, w; cin >> type >> w;
        if (type == "search") cout << "search " << w << ": " << (tr.search(w) ? "yes" : "no") <<
"\n";
        else cout << "prefix " << w << ": " << tr.countPrefix(w) << "\n";
    }
}
```

**INPUT**

```
4
apple app apply ape
5
search app
search appl
prefix ap
prefix app
prefix b
```

**OUTPUT**

```
search app: yes
search appl: no
prefix ap: 4
prefix app: 3
prefix b: 0
```

**How it works:** `end` marks where a full word finishes ("appl" is only a prefix). `pass` counts how many words go through a node, which answers prefix counts instantly.

**Time:**  $O(\text{word length})$  per operation

## 17 Fenwick tree & segment tree

### Spot it when...

- range sum / min / max queries *and* updates mixed together
- $n$  and  $q$  both up to  $10^5$ , so  $O(n)$  per query is too slow
- count inversions, "how many smaller elements to the left"

### The idea

Prefix sums break when values change. These trees keep partial answers so both updates and queries take  $O(\log n)$ . **Fenwick** (Binary Indexed Tree) is short and great for sums. **Segment tree** handles min, max, gcd and more.

### EXAMPLE Fenwick tree: point add, range sum

Operations: `q l r` prints the sum of `a[l..r]`; `u i v` adds `v` to `a[i]` (0-indexed).

```
struct Fenwick {
    int n; vector<long long> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}
    void add(int i, long long v) { for (i++; i <= n; i += i & -i) t[i] += v; }
    long long sum(int i) { long long s = 0; for (i++; i > 0; i -= i & -i) s += t[i]; return s; }
    // a[0..i]
    long long sum(int l, int r) { return sum(r) - (l ? sum(l - 1) : 0); }
};

int main() {
    int n, q; cin >> n >> q;
    Fenwick f(n);
    for (int i = 0; i < n; i++) { long long x; cin >> x; f.add(i, x); }
    while (q--) {
        char op; int a, b; cin >> op >> a >> b;
        if (op == 'q') cout << f.sum(a, b) << "\n";
        else f.add(a, b);
    }
}
```

#### INPUT

```
5 3
1 2 3 4 5
q 0 4
u 2 10
q 1 3
```

#### OUTPUT

```
15
19
```

**How it works:** `i & -i` is the lowest set bit of `i`. Each index stores the sum of a block whose size is that bit, so any prefix is covered by at most  $\log n$  blocks.

**Time:**  $O(\log n)$  per operation

**EXAMPLE Segment tree: point update, range minimum**

Operations: `q l r` prints the minimum of `a[l..r]`; `u i v` sets `a[i] = v`.

```
struct SegTree {
    int n; vector<int> t;
    SegTree(vector<int>& a) : n(a.size()), t(4 * a.size()) { build(a, 1, 0, n - 1); }
    void build(vector<int>& a, int node, int l, int r) {
        if (l == r) { t[node] = a[l]; return; }
        int m = (l + r) / 2;
        build(a, 2 * node, l, m); build(a, 2 * node + 1, m + 1, r);
        t[node] = min(t[2 * node], t[2 * node + 1]);
    }
    void update(int node, int l, int r, int i, int v) {
        if (l == r) { t[node] = v; return; }
        int m = (l + r) / 2;
        if (i <= m) update(2 * node, l, m, i, v); else update(2 * node + 1, m + 1, r, i, v);
        t[node] = min(t[2 * node], t[2 * node + 1]);
    }
    int query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return INT_MAX; // no overlap
        if (ql <= l && r <= qr) return t[node]; // fully inside
        int m = (l + r) / 2;
        return min(query(2 * node, l, m, ql, qr), query(2 * node + 1, m + 1, r, ql, qr));
    }
};

int main() {
    int n, q; cin >> n >> q;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    SegTree st(a);
    while (q--) {
        char op; int x, y; cin >> op >> x >> y;
        if (op == 'q') cout << st.query(1, 0, n - 1, x, y) << "\n";
        else st.update(1, 0, n - 1, x, y);
    }
}
```

**INPUT**

```
6 4
5 3 8 6 1 4
q 0 3
u 1 7
q 0 3
q 2 5
```

**OUTPUT**

```
3
5
1
```

**How it works:** each node stores the minimum of its range; children split the range in half. A query visits only  $O(\log n)$  nodes. To make it a sum tree, change `min` to `+` and `INT_MAX` to `0`.

**Time:**  $O(\log n)$  per operation

## 18 Maths for coding tests

### Spot it when...

- "print the answer modulo  $10^9 + 7$ "
- primes, divisors, factorisation
- nCr, counting arrangements
- gcd / lcm of many numbers

### The idea

Learn a handful of tools: fast power, modular inverse (when the modulus is prime), factorial tables for nCr, the sieve, and trial division for factorising.

### EXAMPLE Prime factorisation and number of divisors

Factorise each number and count its divisors.

```
int main() {
    long long x;
    while (cin >> x) {
        long long n = x, divisors = 1;
        cout << x << " =";
        for (long long p = 2; p * p <= n; p++) {           // only up to sqrt(n)
            if (n % p) continue;
            int e = 0;
            while (n % p == 0) { n /= p; e++; }
            cout << " " << p << "^" << e;
            divisors *= (e + 1);
        }
        if (n > 1) { cout << " " << n << "^1"; divisors *= 2; } // one prime left over
        cout << "   divisors: " << divisors << "\n";
    }
}
```

#### INPUT

```
360
97
```

#### OUTPUT

```
360 = 2^3 3^2 5^1   divisors: 24
97 = 97^1   divisors: 2
```

**How it works:** if  $n = p_1^a \cdot p_2^b \cdot \dots$ , the number of divisors is  $(a+1)(b+1)\dots$ . For 360:  $4 \cdot 3 \cdot 2 = 24$ .

**Time:**  $O(\sqrt{n})$



**EXAMPLE** **nCr modulo a prime**

Answer many "n choose r" queries modulo  $10^9 + 7$ , for n up to  $10^6$ .

```
const long long MOD = 1e9 + 7;
long long power(long long b, long long e) {
    long long r = 1; b %= MOD;
    while (e) { if (e & 1) r = r * b % MOD; b = b * b % MOD; e >>= 1; }
    return r;
}
int main() {
    const int N = 1000000;
    vector<long long> fact(N + 1), inv(N + 1);
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    inv[N] = power(fact[N], MOD - 2); // Fermat: a^(p-2) is the inverse of a
    for (int i = N; i > 0; i--) inv[i - 1] = inv[i] * i % MOD;
    auto C = [&](int n, int r) -> long long {
        if (r < 0 || r > n) return 0;
        return fact[n] * inv[r] % MOD * inv[n - r] % MOD;
    };
    int q; cin >> q;
    while (q--) { int n, r; cin >> n >> r; cout << "C(" << n << ", " << r << ") = " << C(n, r) <<
    "\n"; }
}
```

**INPUT**

```
3
5 2
10 3
1000 500
```

**OUTPUT**

```
C(5,2) = 10
C(10,3) = 120
C(1000,500) = 159835829
```

**How it works:**  $nCr = n! / (r! (n-r)!)$ . Division doesn't work directly with mod, so multiply by modular inverses instead. Precompute once, then each query is  $O(1)$ .

**Time:**  $O(N)$  setup,  $O(1)$  per query

### EXAMPLE Fast power, modular inverse, gcd and lcm

A quick tour of the maths helpers you'll use most.

```
const long long MOD = 1e9 + 7;
long long power(long long b, long long e, long long m) {
    long long r = 1; b %= m;
    while (e) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r;
}
int main() {
    cout << "2^10 = " << power(2, 10, MOD) << "\n";
    long long inv2 = power(2, MOD - 2, MOD);
    cout << "inverse of 2 = " << inv2 << ", check: " << 2 * inv2 % MOD << "\n";
    vector<long long> a = {12, 18, 24};
    long long g = 0, l = 1;
    for (long long x : a) { g = gcd(g, x); l = l / gcd(l, x) * x; } // divide first to avoid
    overflow
    cout << "gcd = " << g << ", lcm = " << l << "\n";
}
```

#### OUTPUT

```
2^10 = 1024
inverse of 2 = 500000004, check: 1
gcd = 6, lcm = 72
```

**How it works:** fast power squares the base and halves the exponent each step, so  $a^b$  takes  $O(\log b)$  multiplications.

## 19 Quick revision tables

### Pattern → typical time

| Pattern                 | Typical time                    | Classic problems to revise                                    |
|-------------------------|---------------------------------|---------------------------------------------------------------|
| Two pointers            | $O(n)$ after $O(n \log n)$ sort | pair sum, 3Sum, container with most water                     |
| Sliding window          | $O(n)$                          | longest substring without repeats, min subarray $\geq$ target |
| Prefix sums + hash map  | $O(n)$                          | subarray sum = k, divisible by k                              |
| Binary search on answer | $O(n \log \text{range})$        | Koko bananas, aggressive cows, ship packages                  |
| Monotonic stack         | $O(n)$                          | next greater, daily temperatures, histogram                   |
| Greedy                  | $O(n \log n)$                   | activity selection, platforms, jump game                      |
| Backtracking            | $O(2^n)$ or $O(n!)$             | subsets, permutations, N-Queens                               |
| Tree DFS                | $O(n)$                          | height, diameter, LCA, validate BST                           |
| BFS / DFS               | $O(V + E)$                      | islands, rotting oranges, components                          |
| Topological sort        | $O(V + E)$                      | course schedule, build order                                  |
| Dijkstra                | $O((V + E) \log V)$             | network delay, cheapest path                                  |
| Union-find              | $\approx O(1)$ per op           | redundant connection, Kruskal                                 |
| 1D / 2D DP              | $O(n)$ to $O(n \cdot m)$        | house robber, knapsack, LCS, edit distance                    |
| Interval / bitmask DP   | $O(n^3)$ / $O(2^n n^2)$         | matrix chain, burst balloons, TSP                             |
| Heap                    | $O(n \log k)$                   | top K, median stream, merge K lists                           |
| Fenwick / segment tree  | $O(\log n)$ per op              | range sum/min with updates, inversions                        |

### Mistakes that cost marks in DSA problems

1. **Using sliding window when numbers can be negative.** Shrinking no longer lowers the sum; use prefix sums + hash map.
2. **Binary search that never ends.** When you set `lo = mid`, compute mid rounding up: `lo + (hi - lo + 1) / 2`.
3. **Greedy without proof.** Try a small counter-example before trusting it (e.g. 0/1 knapsack is not greedy).
4. **Forgetting to undo the choice in backtracking** (missing `pop_back()` or `used[i] = false`).
5. **Not marking nodes visited when they're pushed in BFS**, so the same node enters the queue many times.
6. **Recursion depth:** DFS on a  $10^6$ -node line graph can crash. Use BFS or your own stack.
7. **Wrong loop direction in knapsack:** downwards for 0/1, upwards for unbounded.
8. **Validating a BST by comparing a node only with its children.** Pass down (low, high) bounds.
9. **Dijkstra with negative edges.** It gives wrong answers; use Bellman–Ford.
10. **Floyd–Warshall with k as an inner loop.** k must be the outermost loop.
11. **Overflow in DP counts.** Use long long, and take the modulo at every step when asked.
12. **Off-by-one in prefix sums.** Use pre of size  $n + 1$  with `pre[0] = 0`, then `sum(l..r) = pre[r+1] - pre[l]`.

*Revise one section a day. For each example, cover the code, read only the problem, and try to write it yourself in 15 minutes. Then compare.*